

Pensieve: An Audit-Verifiable Retrieval and Numeric-Grounding Architecture for Financial Filings

Apurva Malasi

University School of Automation and Robotics

Abstract

We present the design, implementation, and deployment of Pensieve, a retrieval-augmented generation (RAG) system for financial filings — annual reports, 10-K-style disclosures, and investor prospectuses — built around a single governing constraint: every numeric figure the system states must be programmatically traceable to a specific cell in the source document's extracted table grid before it is returned to the user. Standard RAG pipelines treat a document's content uniformly as flat text, which is adequate for narrative question answering but structurally unsound for numeric question answering, since a language model asked to read a financial table from serialized prose has no mechanism to prevent it from stating a plausible but incorrect figure. Pensieve addresses this by preserving document structure end to end: IBM Docling parses each filing into narrative text blocks and table grids with page-level provenance; a heuristic risk flagger scores each extracted table for structural extraction quality; a dual-path retrieval layer routes narrative and numeric queries to a content-type- matched retrieval strategy; and a dedicated numeric-grounding stage regenerates every stated figure and checks it, character-for-character after normalization, against the actual retrieved table cells before the answer is returned. The verification outcome is reported to the user as one of four explicit states — `verified`, `verified_low_confidence`, `unverified`, or `not_applicable` — rather than being collapsed into a single undifferentiated answer. The system is deployed as a full-stack web application (a Next.js 16 / React 19 / TypeScript frontend and a Python/FastAPI backend, both on Render, backed by Qdrant Cloud) and serves a live indexed corpus of five financial filings totaling 2,224 chunks. This report documents the architecture, the specific engineering decisions made to keep the system honest about its own uncertainty rather than merely fluent, the deployment constraints encountered on a memory-limited hosting tier, and the following the evaluation discipline this report is modeled on an explicit account of what the current validation evidence does and does not establish about the system's reliability at scale.

1. Introduction

Financial analysts, equity researchers, and compliance reviewers routinely need to extract specific, exact figures from long, table-dense disclosure documents — a task that is simultaneously tedious (a 190-page annual report may contain over 200 tables) and unforgiving of error (a misread figure in an investment or compliance context has real consequences). Large language models have made it trivially easy to ask a document a question and receive a fluent answer. Fluency, however, is not the same property as fidelity: a model asked to state a company's net interest income from a retrieved passage of serialized table text has no built-in mechanism preventing it from stating a plausible-sounding number that does not actually appear in the source. Worse, a confidently stated wrong number is more dangerous than an admitted absence of an answer, because its fluent delivery suppresses the reader's ordinary skepticism.

Standard RAG architectures do not solve this. They mitigate outright hallucination by grounding generation in retrieved passages, but they provide no post-hoc check that a specific numeric claim in the

output actually corresponds to a value in the retrieved source, grounding in text is not the same guarantee as verification against structured data. For narrative questions ("what does management say about credit risk"), this gap is tolerable, since narrative fidelity is not straightforwardly checkable by a program in the first place. For numeric questions ("what were total deposits as of December 31, 2024"), it is not tolerable, because exact verification against the source is both possible and, for this class of question, the entire point.

Pensieve is built around one governing design principle, applied uniformly: **every numeric claim the system makes must be checked against an independently extracted, structured representation of its claimed source before it is shown to the user, and the outcome of that check must be disclosed, not hidden.** This report documents the resulting architecture, the specific failure modes each design decision was chosen to prevent, and consistent with the evaluation discipline of the report this document is modeled on an explicit statement of what the current evidence does and does not establish, rather than treating a working demo as a substitute for a rigorous evaluation.

2. Related Work

2.1 Retrieval-Augmented Generation

The canonical RAG architecture (Lewis et al., 2020) retrieves the top-k passages relevant to a query from a vector index and conditions generation on them. Subsequent work has refined retrieval quality — hybrid dense/sparse retrieval, re-ranking, multi-hop retrieval but the underlying unit of retrieval, an undifferentiated text passage, has remained largely unchanged. This abstraction is adequate for prose-dominant corpora. It is not adequate for financial filings, where the authoritative figures live in structured tables that prose merely paraphrases in rounded or approximate form, and where a fixed-size chunking window will, without special handling, split a table mid-row and sever a value from the row label that gives it meaning.

2.2 Document Layout and Table Structure Recognition

Document-understanding models such as Docling (IBM Research) apply layout analysis and table-structure recognition to recover a PDF's semantic structure, distinguishing headings and paragraphs from tables, and extracting table cells into row-column grids rather than flattened text. Pensieve treats Docling as a necessary prerequisite, not an end-to-end solution: Docling's own extraction is not perfectly reliable on complex, multi-row-header financial tables (Section 6.3), and the retrieval, chunking, and verification machinery built on top of it exists specifically to make the rest of the pipeline honest about that imperfection rather than to assume it away.

2.3 Answer Verification

Post-hoc verification of model output has precedent in mathematical and logical reasoning (chain-of-thought verification, self-consistency sampling), but these approaches typically verify internal consistency, not correspondence to an external structured source. Pensieve's verification stage (Section 5) is a domain-specific instantiation of a different, stronger property: not "is this answer self-consistent," but "does this specific stated number exist, verbatim after normalization, in the specific table cell the answer claims it came from."

3. Design Philosophy and Operational Trade-offs

Several decisions precede any discussion of individual modules, because they determine what the rest of the system is trying to guarantee.

3.1 Content-Type-Aware Chunking, Not Uniform Chunking

The naive alternative of treat all extracted content as flat text and chunk it uniformly by token count was rejected. Narrative text and tabular data have fundamentally different retrieval and grounding semantics: a narrative passage's meaning degrades gracefully if a paragraph boundary falls slightly wrong, while a table's meaning does not degrade gracefully at all if a row is severed from its header, because the resulting fragment is not merely incomplete, it is actively misleading if fed to a generation model as though it were complete. Pensieve therefore implements two disjoint chunking strategies (Section 6.1): a sliding window over narrative text, and **exactly one chunk per table, never split**, regardless of the table's size.

3.2 Deterministic Cell Matching, Not LLM Self-Verification

A tempting shortcut is to ask the same (or a second) LLM to check its own generated figure against the retrieved context — "does the number you just stated actually appear above." This was deliberately rejected. An LLM asked to verify its own arithmetic or its own reading of a table is subject to exactly the same failure mode that produced the original risk: it can confidently affirm a match that is not actually present. Pensieve's verification stage instead performs deterministic, programmatic string matching (with numeric normalization) between the generated answer's extracted figures and the raw grid cells of the retrieved table chunks, a mechanism that, unlike a second LLM call, cannot hallucinate agreement.

3.3 Decoupled Ingestion and Serving

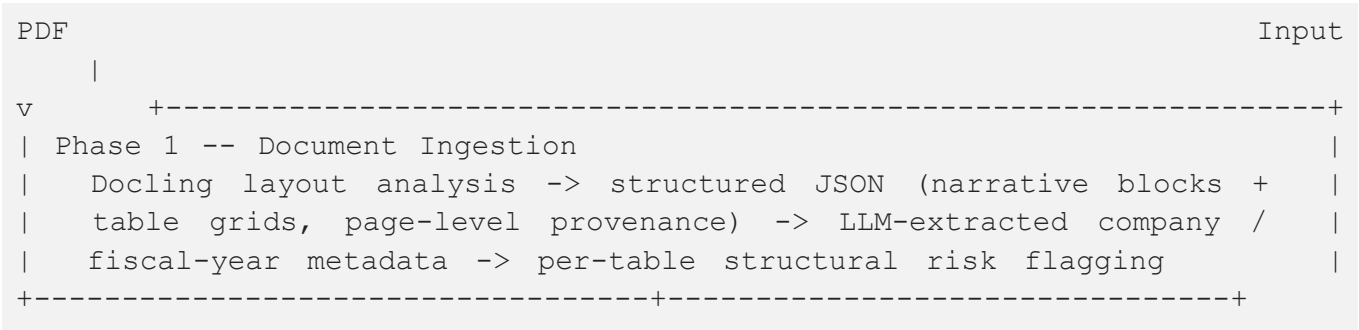
Ingestion (Docling parsing, risk flagging, chunking, embedding) and serving (retrieval, generation, verification) communicate exclusively through immutable, content-hashed JSON artifacts, not through a shared in-memory pipeline. This means a change to the serving layer never requires re-running ingestion, ingestion can be re-run idempotently (identical input produces identical chunk IDs, so re-ingestion is a safe no-op on unchanged content), and the heavier ingestion dependencies (Docling, PyTorch) can be excluded from the serving process entirely in constrained deployment modes (Section 7.2).

3.4 Fail Toward Rigor, Not Toward Fluency

Where the system is uncertain, it is designed to default toward the more conservative, verification-bearing path rather than the more fluent one. The query-intent classifier (Section 4.3) is explicitly biased to default to "numeric" on any ambiguity or failure, on the stated reasoning that misclassifying a narrative query as numeric costs one redundant retrieval call, while misclassifying a numeric query as narrative produces an answer with no verification claim at all on a question where the user specifically needed one.

4. System Architecture

Pensieve is implemented as a sequential pipeline of independently testable Python modules (Appendix C), deployed behind a FastAPI backend and a Next.js 16 / React 19 / TypeScript frontend.



```

|
v +-----+
| Phase 2 -- Chunking, Embedding, and Indexing |
| Narrative: sliding window (400 tok target, 500 tok max, |
| 50 tok overlap, cl100k_base) |
| Tables: exactly one chunk per table, never split |
| Embeddings: BAAI/bge-small-en-v1.5, 384-dim, local, CPU |
| Index: Qdrant Cloud, cosine similarity, payload-indexed on |
| company_name / fiscal_year / section_type / risk_flag |
+-----+
|
v +-----+
| Phase 3 -- Intent Classification and Narrative Retrieval |
| LLM classifies query -> "narrative" | "numeric" (bias: numeric |
| on ambiguity or failure) |
| Narrative path: top-6 text chunks, similarity >= 0.45, |
| cited generation with hallucinated-citation stripping |
+-----+
|
v +-----+
| Phase 4 -- Numeric Grounding and Verification |
| Numeric path: top-3 table chunks, 6,000-token context budget |
| (whole tables dropped under budget pressure, never truncated) |
| LLM generates from raw table grids |
| Every stated figure programmatically matched against source |
| cell values -> verified / verified_low_confidence / unverified |
+-----+
|
v +-----+
| Phase 5 -- Multi-Document Comparison |
| LLM extracts comparison entities (company, fiscal year) |
| Per-entity: independent Phase 3/4 pipeline execution |
| Deltas computed programmatically, never by LLM arithmetic |
| Overall status: weakest-link rollup across entities |
+-----+
|
v +-----+
| Phase 6 -- API and Frontend |
| Backend: FastAPI on Render |
| Frontend: Next.js 16 / React 19 / TypeScript on Render |
| Async ingestion jobs, per-IP rate limiting, showcase mode |
+-----+

```

4.1 Document Ingestion

structurer.py accepts a PDF path and returns a canonical structured JSON representation:

```

{ "doc_id": "sha256:...", "source_filename": "annual_report_2024.pdf",
  "text_blocks": [ {"page": 2, "type": "heading", "level": 1, "text":

```

```
"Revenue Summary"}, {"page": 2, "type": "paragraph", "text": "The company achieved..."} ], "tables": [ { "table_id": "t_0002_01", "page_start": 2, "page_end": 2, "grid": [ ["", "FY2024", "FY2023"], ["Revenue", "12,450", "11,230"] ], "risk_flag": false, "risk_reasons": [] } ] }
```

`text_blocks` feed the narrative chunking path; `tables` are preserved as row-column grids specifically for the numeric verification engine described in Section 5. A second module, `metadata_extractor.py`, sends the first three pages of each filing (typically under 4,000 characters) to an LLM with a tightly constrained extraction prompt, returning `company_name` and `fiscal_year` as structured JSON. These two fields are attached to every chunk derived from the document as Qdrant payload metadata, which is what makes the filtered, per-entity retrieval used in Phase 5's comparison engine possible.

4.2 Structural Risk Flagging

Docling's table-structure recognition is not perfectly reliable on financial filings specifically: complex, multi-row-header tables can be merged or scrambled during extraction. `risk_flagger.py` scores every extracted table against three independent structural signals (Table 1) before it is indexed.

Signal	Condition	Failure Mode It Detects
Repeated header phrase	A multi-word phrase recurs ≥ 2 times within a single row	Docling merged two physically separate tables at a shared column header
Row-label duplication	(row count / distinct leading-column labels) > 2.5 , with ≥ 3 distinct labels	Two tables with overlapping row labels were concatenated into one
Multi-value cells	$\geq 30\%$ of rows contain a cell holding ≥ 2 numeric tokens	Unsplit column headers or merged values embedded as a single cell

Table 1. Structural risk-flagging signals and the extraction failure mode each is designed to catch.

A flagged table is not suppressed or excluded from retrieval — its content is used exactly as any other table's would be. The flag instead propagates as chunk metadata into the verification stage, where it downgrades an otherwise-matching figure's confidence label from `verified` to `verified_low_confidence` (Section 5). This is a deliberate design choice: a structurally questionable table is still often the only available source for a figure, and silently excluding it would be a worse outcome for the user than surfacing the figure with an honest caveat.

4.3 Query Intent Classification

Every incoming query is classified as `"narrative"` or `"numeric"` by a small, tightly constrained LLM call before retrieval begins. Consistent with the fail-toward-rigor principle (Section 3.4), the classifier defaults to `"numeric"` on any ambiguous output, empty response, or LLM call failure — on the stated reasoning that the cost of misclassifying in the numeric direction (one redundant table retrieval) is strictly lower than the cost of misclassifying in the narrative direction (a numeric question answered with no verification claim at all).

5. The Numeric Grounding and Cell-Verification Engine

This is the system's central technical contribution and the component that most directly implements the governing design principle stated in Section 1.

5.1 Table Retrieval and Context Budgeting

For a query classified "numeric", retrieval is filtered to `section_type = "table"` chunks only, with a smaller top-k of 3 (against 6 for narrative retrieval) — whole tables are large, and a broad top-k risks context-window saturation on a question that is typically well answered by one or two specific tables. Before generation, `fit_tables_to_context` enforces a 6,000- token budget across the retrieved tables. Under budget pressure, **whole low-scoring tables are dropped, never truncated** — truncating a table mid-row would corrupt the exact cell values the verification stage depends on, so the system is designed to answer from fewer complete tables rather than more incomplete ones.

5.2 Generation from Raw Grids

Retrieved tables are presented to the generation model as their raw row-column grid content, not as pre-summarized prose, and the model is explicitly instructed not to round or reformat figures. This is a deliberate constraint on the generation prompt: reading a value out of a grid and citing the specific cell it came from is a categorically different task from paraphrasing a table's contents, and only the former produces an output the verification stage in Section 5.3 can meaningfully check.

5.3 Programmatic Cell Verification

After generation, `verify_numbers_against_grid` performs the core check, formalized in Appendix A:

1. **Extract** every numeric figure from the generated answer text via regex, handling currency symbols, comma-separated thousands, percentages, and parenthesized negatives.
2. **Normalize** each figure to a canonical form (commas stripped, consistent decimal and sign representation).
3. **Match** each normalized figure against every cell in the grid of the table chunk(s) the answer cites.
4. **Classify** the result into one of four explicit states.

Status	Condition	User-Facing Meaning
<code>verified</code>	All stated figures matched a source cell; cited table carries no risk flag	The figure was checked and its extraction is trusted
<code>verified_low_confidence</code>	All stated figures matched a source cell; cited table carries a risk flag	The figure was checked, but its source table had extraction quality issues — worth a manual glance
<code>unverified</code>	One or more stated figures could not be matched to any retrieved cell	The system could not confirm this figure — treat with caution
<code>not_applicable</code>	The answer contains no numeric figures (e.g., an explicit no-data response)	No verification claim is being made

Table 2. The four-state verification contract.

This four-state contract, rather than a binary verified/unverified split, is itself a design decision: collapsing `verified_low_confidence` into `verified` would overstate confidence in a figure whose source extraction is structurally suspect, while collapsing it into `unverified` would discard a figure that did, in fact, match its cited source — the distinction is the entire point of connecting the ingestion-time risk flag (Section 4.2) to the serving-time verification outcome.

6. Data Engineering and Filing Corpus

6.1 Chunking Strategy

Narrative chunks use a sliding window over `text_blocks` with a target size of 400 tokens (cl100k_base tokenizer), a hard maximum of 500 tokens, and 50-token overlap between consecutive chunks. Headings are attached to the content block that follows them rather than becoming independent chunks, so a retrieved chunk always carries its heading context. Blocks shorter than a minimum meaningful-content threshold (e.g., isolated page numbers) are merged into adjacent chunks rather than indexed as standalone noise.

Table chunks are never split: each extracted table becomes exactly one chunk regardless of size, per the reasoning in Section 3.1. Tables whose rendered content exceeds the local embedding model's token limit have their *embedding text* truncated for search-matching purposes only, the full, untruncated grid is always preserved in the Qdrant payload and is what the verification engine (Section 5.3) actually operates against.

Chunk IDs are deterministic content hashes of the document ID, page range, and content prefix, which makes re-ingestion idempotent: re-running ingestion on an unchanged document produces identical chunk IDs, so the corresponding Qdrant upserts are safe no-ops.

6.2 Embedding and Indexing

Pensieve defaults to `BAAI/bge-small-en-v1.5` via `sentence-transformers` — a 384-dimensional model that runs on CPU with no GPU dependency, a deliberate choice driven by the deployment target (Section 7) rather than by embedding quality alone. The embedding call is abstracted behind a single provider-agnostic function signature, so switching to a larger hosted model (e.g., a 1,536-dimensional OpenAI embedding) is a configuration change; the Qdrant collection's vector dimension is derived from the active provider at collection-creation time, not hardcoded.

The vector store client includes production-hardening measures added after observing real failure conditions on Qdrant Cloud's free tier: a 120-second connection timeout to prevent silent hangs on network partitions, 32-chunk upsert batching (reduced from an initial 64 after hitting payload size limits), exponential-backoff retry on upsert failure, and an explicit client-refresh capability to recover from stale connections after a hosting-platform sleep/wake cycle.

6.3 Filing Corpus

At the time of writing, the deployed index contains **2,224 chunks** across **five** financial filings. Table 3 relates directly to the risk-flagging mechanism described in Section 4.2: it is the actual before/after regression check run against a recalibration of the risk heuristic, using known true- and false-positive tables identified through manual inspection of a real filing (Republic Bancorp, FY2024) during development.

Check	Result
Flag rate before recalibration	100 / 204 tables flagged (49.0%)

Check	Result
Flag rate after recalibration	67 / 204 tables flagged (32.8%)
Known true positive (page 124, multi-row credit-quality table)	Correctly remained flagged after recalibration
Known true positive (page 89, rate-sensitivity table)	Correctly remained flagged after recalibration
Known false positive (page 7, glossary table)	Correctly cleared after recalibration
Known false positive (page 25, capital-ratio table)	Correctly cleared after recalibration
Known false positive (page 44, stock-performance table)	Correctly cleared after recalibration

Table 3. Risk-flagger recalibration regression check, verified against manually inspected ground truth on one real filing.

The original, uncalibrated heuristic flagged tables based solely on repeated phrases anywhere in the table, which produced a high false-positive rate on tables whose *header* row legitimately repeats sub-column labels (e.g., "Amount"/"Ratio" across two fiscal years). The recalibrated heuristic excludes header rows from this specific check, which is what produced the drop from 49.0% to 32.8% without losing either known true positive.

7. Cloud Deployment and Memory Architecture

7.1 Deployment Topology

The backend (FastAPI) and frontend (Next.js 16 / React 19 / TypeScript) are deployed as separate Render services from a single monorepo, each pointed at its respective subdirectory as the platform build root. Qdrant runs as a separate managed Qdrant Cloud cluster, decoupled entirely from the application's deployment lifecycle. The LLM provider is NVIDIA NIM's OpenAI-compatible API, used for metadata extraction, query classification, and both narrative and numeric answer generation.

7.2 Memory Optimization Under a Constrained Hosting Tier

Render's hosting tier used for this deployment enforces a hard memory ceiling, and a Python process importing `docling`, `torch`, and `sentence-transformers` eagerly at module load time was found to consume roughly 401 MB of resident memory before serving a single request — leaving almost no headroom under the platform's limit and causing intermittent out-of-memory process kills. Two changes brought startup memory down to approximately 148 MB, a 63% reduction:

- **Lazy imports inside background-worker functions.** Heavy dependencies (`docling`, `torch`, `sentence_transformers`) are imported inside the functions that actually invoke them rather than at module scope. Python's import cache means the cost is paid once, on the first request that exercises that code path, rather than unconditionally at process startup.
- **Showcase mode.** A `SHOWCASE_MODE` environment variable, when enabled, rejects the ingestion-related endpoints (`POST /documents/upload`, `DELETE /documents/{doc_id}`) with an immediate 403 before any of their dependencies are touched — meaning the heaviest dependencies (Docling's OCR and layout models) are never imported for the lifetime of a showcase-mode process, permanently reclaiming the memory they would otherwise have consumed.

7.3 Public-Facing Deployment Mode

The live deployment runs in showcase mode: a fixed catalog of five pre-ingested filings, no public upload or delete capability, and per-IP rate limiting of 15 requests/minute on `/ask` and `/compare`. This mode exists for two independent reasons, not one — the memory savings described above, and a deliberate reduction of public attack surface, since an open upload endpoint on a small hosted backend is both a cost risk (uncontrolled LLM API usage) and an availability risk (a large PDF's OCR pass can occupy the process for an extended period). Ingestion and delete functionality remain fully implemented and available in local development (`SHOWCASE_MODE=false`), so extending the live catalog requires no code changes, only running the existing pipeline locally against the same Qdrant Cloud cluster and redeploying.

8. Results and Empirical Validation

Consistent with the evaluation discipline this report is modeled on, this section is explicit about the difference between what has been verified through targeted, documented test cases and what would require a larger-scale quantitative study that has not yet been performed (Section 9).

8.1 Targeted Verification Test Cases

Table 4 reports four live end-to-end test cases run against the real deployed retrieval, generation, and verification pipeline (not mocked), using the Republic Bancorp FY2024 filing.

Test Case	Query	Result
Clean table, expect <code>verified</code>	"What were total traditional bank deposits as of December 31, 2024?"	<code>verified</code> — figure matched a source cell in a table with no risk flag
Flagged table, expect <code>verified_low_confidence</code>	"What is the impact of a 400 basis point rate change on net interest income?"	<code>verified_low_confidence</code> — figure matched a source cell, but the cited table (page 89) carries a risk flag
No matching data, expect <code>no_context_found</code>	An obscure numeric query with no corpus coverage	<code>no_context_found: true</code> — no fabricated answer produced
Intent classification spot-check	10 varied real queries across narrative and numeric phrasing	10/10 classified correctly against manual expectation

Table 4. Live, non-mocked verification test cases against the deployed pipeline.

8.2 Risk Flagger Calibration

Reported in full in Table 3 (Section 6.3): the recalibrated risk heuristic reduced the false-positive flag rate from 49.0% to 32.8% on a real filing while preserving both known true positives identified through manual inspection.

8.3 What Has Not Been Measured

No large-scale quantitative benchmark — a fixed set of held-out question/answer pairs scored for precision, recall, or hallucination catch-rate across the full corpus — has been run against Pensieve. Nor has retrieval or generation latency been formally profiled under production-representative load. This is stated explicitly here, rather than left implicit, and is identified as the top empirical priority in Section 10.

9. Discussion and Limitations

The evaluation performed so far is targeted, not comprehensive. Section 8's test cases confirm that the four-state verification contract behaves correctly on specific, manually identified cases — a clean table, a known-flagged table, an absent-data case, and a classification spot-check — but this is not equivalent to a statistically powered claim about precision or recall across the full space of financial-filing question types. A reader should not infer a general accuracy rate from four passing test cases; that inference is exactly the gap Section 8.3 and Section 10 flag as unresolved.

The risk-flagging heuristic is rule-based, not learned, and was calibrated on one filing. The three signals in Table 1 were derived by manually inspecting Docling's actual failure modes on a single real annual report. They generalize plausibly to other financial filings with similar table conventions, but this has not been independently verified against the other four filings in the current corpus, nor against filing types (e.g., IPO prospectuses) with potentially different table structures.

Verification proves consistency with the extracted grid, not correctness of the extraction itself. This distinction is the entire reason the `verified_low_confidence` state exists rather than collapsing to a binary outcome (Section 5.3), but it is worth restating plainly: a `verified` result means the stated figure matches what Docling extracted, not that Docling's extraction is itself a perfect transcription of the source PDF. A structurally clean-looking table can still, in principle, contain an extraction error that the risk heuristic did not catch — the heuristic is a calibrated signal, not a guarantee.

The classifier's numeric-bias default is untested at the margin. The fail-toward-rigor policy (Section 3.4) is a reasoned default, not an empirically tuned one — no experiment has been run comparing it against an unbiased classifier to confirm the assumed cost asymmetry actually holds across a representative query distribution, as opposed to the small classification spot-check reported in Table 4.

Memory optimization was validated by observed process behavior, not formal profiling. The 401 MB to 148 MB figures reported in Section 7.2 were obtained by observing the deployed process's resident memory before and after the lazy-import and showcase-mode changes, not through a controlled, repeated memory-profiling harness. The relative improvement is real and directly resolved the platform's out-of-memory failures, but the exact figures should be read as point observations rather than statistically characterized measurements.

10. Conclusion and Future Work

Pensieve demonstrates that structural fidelity — preserving the distinction between narrative and tabular content from ingestion through to the final answer — makes programmatic numeric verification a tractable addition to a RAG pipeline, rather than a hypothetical one. The four- state verification contract (Section 5.3), the never-split table chunking rule (Section 3.1 / 6.1), and the programmatic (not LLM-based) delta computation used in multi-document comparison are the specific mechanisms that turn "grounded in retrieved text" into the stronger claim "the specific number stated was checked against its claimed source."

The most valuable next steps, in priority order:

1. **A held-out, quantitatively scored evaluation set** — a fixed collection of question/answer pairs with known-correct figures, scored for precision, recall, and `false-unverified` rate across the full corpus, to replace the targeted test cases in Section 8 with a statistically grounded claim.

2. **Cross-filing validation of the risk-flagging heuristic**, since it was calibrated on a single filing (Section 9) and its generalization to the corpus's other four filings, and to structurally different document types such as IPO prospectuses, has not been independently confirmed.
3. **Formal latency and memory profiling under representative concurrent load**, rather than the point observations reported in Section 7.2, particularly given the constrained hosting tier's sensitivity to memory pressure.
4. **A cross-encoder re-ranking stage** for retrieval, motivated by the observed limitation that questions whose answer spans multiple non-contiguous document sections are not well served by top-k vector similarity alone.
5. **Expansion of the ingested corpus beyond the current five filings** toward a scale sufficient to support the quantitative evaluation described in item 1.

References

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*.

IBM Research. Docling: Document Conversion and Understanding Toolkit. <https://github.com/DS4SD/docling>

Qdrant. Qdrant Vector Database Documentation. <https://qdrant.tech/documentation/>

NVIDIA. NIM: NVIDIA Inference Microservices API Documentation. <https://docs.nvidia.com/nim/>

Touvron, H. et al. (2023–2024). Llama: Open Foundation and Fine-Tuned Language Models. Meta AI.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Appendix A: The Cell-Verification Formal Protocol

This appendix specifies the algorithm behind Section 5.3's `verify_numbers_against_grid`.

```
function verify_numbers_against_grid(answer_text, cited_table_chunks):
    figures <- extract_numeric_figures(answer_text) # regex: integers,
    decimals, comma-separated thousands, # percentages, parenthesized
    negatives, currency-prefixed values

    if figures is empty: return VerificationResult(status = NOT_APPLICABLE)

    all_matched <- true any_risk_flag <- false

    for each figure in figures: normalized <- normalize(figure) # strip commas,
    unify decimal/sign representation

    matched <- false for each chunk in cited_table_chunks: for each cell in
    chunk.grid: if normalize(cell) == normalized: matched <- true if
    chunk.risk_flag == true: any_risk_flag <- true break

    if matched == false: all_matched <- false

    if all_matched == false: return VerificationResult(status = UNVERIFIED)
```

```

else if any_risk_flag == true: return VerificationResult(status =
VERIFIED_LOW_CONFIDENCE) else: return VerificationResult(status = VERIFIED)

```

Two properties of this protocol are load-bearing and worth stating explicitly. First, matching is performed against the cited table chunk(s) specifically, not against the entire retrieved context — a coincidental match in an unrelated table is not treated as verification. Second, the protocol is purely deterministic string/numeric comparison; no LLM call participates in the verification decision itself, consistent with the rationale in Section 3.2.

Appendix B: Reference Operator Interface

The Next.js 16 / React 19 / TypeScript frontend presents the verification contract described in Section 5.3 directly to the end user rather than only in API responses. The interface consists of a document catalog panel — listing each ingested filing with its page count, total extracted table count, and the count of tables carrying a structural risk flag, giving the user an upfront signal of a given filing's overall extraction reliability before they ask anything — and a central chat interface in which every answer to a numeric query carries one of the four verification states from Table 2, rendered with a distinct visual treatment per state rather than a single generic "answer" presentation. Citations are rendered as inspectable elements: for a table-sourced answer, expanding a citation reveals the actual cited table content and, if applicable, the specific risk-flag reason recorded during ingestion (Section 4.2) — surfacing the disclaimer at the exact point where the user is evaluating the source, not only as a detached badge. Cross-document comparison answers (Phase 5) are rendered with each entity's result shown individually alongside the synthesized comparison, so that a comparison's overall confidence is never collapsed into a single figure without a way to inspect which specific side of the comparison is the weaker one. The frontend's development process included iterative visual review via automated browser inspection at each major interface change, rather than relying on code review of styling changes alone. A published screenshot of a pre-final iteration of this interface exists from development; it is not reproduced here, as the interface's visual design has since been substantially revised, and an outdated image would misrepresent the current deployed state.

Appendix C: Module Reference

Module	Phase	Responsibility
structurer.py	1	PDF to structured JSON via Docling
metadata_extractor.py	1	Company name and fiscal year extraction via LLM
converter.py	1	Docling document object to canonical JSON format
risk_flagger.py	1	Table structural risk assessment
chunker.py	2	Narrative windowing and table-as-single-chunk logic
embedder.py	2	Embedding provider abstraction (local / hosted)
embed_runner.py	2	Batch embedding and Qdrant upsert orchestration

Module	Phase	Responsibility
<code>vector_store.py</code>	2	Qdrant client lifecycle, retry, and timeout management
<code>classifier.py</code>	3	Query intent classification (narrative / numeric)
<code>retriever.py</code>	3, 4	Type-filtered Qdrant retrieval for both retrieval paths
<code>generator.py</code>	3	Narrative RAG generation and citation parsing
<code>numeric_grounding.py</code>	4	Table-grounded generation, figure extraction, verification
<code>comparator.py</code>	5	Multi-document entity extraction and comparison orchestration
<code>api.py</code>	6	FastAPI endpoints, async ingestion jobs, rate limiting
<code>config.py</code>	all	Centralized constants and environment-variable resolution
<code>batch_runner.py</code>	all	CLI orchestration for bulk ingestion
<code>logger.py</code>	all	Structured logging configuration

Appendix D: Deployment Configuration

Parameter	Value
Backend platform	Render
Frontend platform	Render
Frontend stack	Next.js 16, React 19, TypeScript
Startup memory (post-optimization)	~148 MB
Startup memory (pre-optimization)	~401 MB
Embedding model	BAAI/bge-small-en-v1.5 (384-dim, CPU, local)
LLM provider	NVIDIA NIM (OpenAI-compatible API)
Metadata / classification LLM	meta/llama-3.1-70b-instruct
Generation LLM	meta/llama-3.2-11b-vision-instruct
Vector database	Qdrant Cloud (cosine similarity)
Indexed chunks	2,224 across 5 filings
Public deployment mode	Showcase (SHOWCASE_MODE=true; upload/delete disabled)
Rate limit	15 requests/minute per IP on /ask and

Parameter	Value
	/compare